# Lecture 10. Splines and Generalized Additive Models

Polynomial regression, regression and smoothing splines, local regression, GAMs

## Contents


**10.1 Beyond linearity** **2**

**10.2 Polynomial regression and step functions** **2**

**10.3 Regression splines** **3**

**10.4 Smoothing splines** **5**

**10.5 Local regression** **6**

**10.6 Generalized additive models** **6**

## 10.1 Beyond linearity

**Motivation.** Linear models are simple to fit and interpret, and regularization improves them further, but some relationships are clearly curved. The methods here relax linearity one step at a time while keeping as much of the linear model's interpretability as possible.

Linear models are simple to describe, implement and interpret, and the linearity assumption is often a good approximation. Regularization (Lecture 7) reduces their variance. To relax linearity itself while staying interpretable we consider polynomial regression and step functions, regression and smoothing splines, local regression, and generalized additive models (GAMs). The running example is `ISLR2::Wage`: wages of 3000 male workers in the US Mid-Atlantic region.

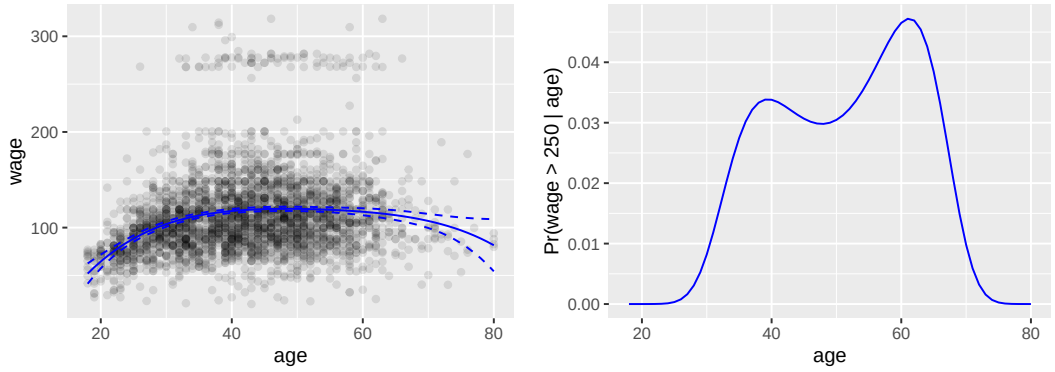
## 10.2 Polynomial regression and step functions

**Polynomial regression** replaces the line by

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \epsilon_i,$$

still linear in the coefficients, hence fitted by least squares. Degrees above 4 are rarely used: the curve becomes overly flexible, with strange shapes near the boundary of $X$. The individual coefficients are not of interest; we look at the fitted function $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \cdots + \hat{\beta}_4 x_0^4$ and its pointwise confidence band ($\pm 2$ standard errors).

```
fit4 <- lm(wage ~ poly(age, 4), data = Wage)
grid <- tibble(age = 18:80)
pr <- predict(fit4, grid, se.fit = TRUE)
grid <- grid %>% mutate(fit = pr$fit, lo = fit - 2 * pr$se.fit, hi = fit + 2 * pr$se.fit
    )
ggplot(Wage, aes(age, wage)) + geom_point(alpha = 0.1) +
  geom_line(data = grid, aes(y = fit), colour = "blue") +
  geom_line(data = grid, aes(y = lo), colour = "blue", linetype = "dashed") +
  geom_line(data = grid, aes(y = hi), colour = "blue", linetype = "dashed")
fit_log <- glm(I(wage > 250) ~ poly(age, 4), family = binomial, data = Wage)
grid %>% mutate(p = predict(fit_log, grid, type = "response")) %>%
  ggplot(aes(age, p)) + geom_line(colour = "blue") + labs(y = "Pr(wage > 250 | age)")
```




Wages seem to come from two populations, a high-earner group above \$250 000 and the rest. Treating $\mathbb{1}(\text{wage} > 250)$ as binary, logistic regression with a polynomial in age models

$$P(y_i > 250 \mid x_i) = \frac{\exp(\beta_0 + \beta_1 x_i + \cdots + \beta_d x_i^d)}{1 + \exp(\beta_0 + \beta_1 x_i + \cdots + \beta_d x_i^d)};$$

its confidence intervals (not drawn) are very wide because few people are high earners.

**Step functions** avoid imposing a global structure. Cut the range of $X$ at $c_1 < \cdots < c_K$ into $K+1$ bins and use the indicators

$$C_0(X) = I(X < c_1),\ C_1(X) = I(c_1 \le X < c_2),\ \ldots,\ C_K(X) = I(c_K \le X),$$

which sum to 1 for every $X$. Fit $y_i = \beta_0 + \beta_1 C_1(x_i) + \cdots + \beta_K C_K(x_i) + \epsilon_i$. Then $\beta_0$ is the mean of $Y$ for $X < c_1$, $\beta_0 + \beta_j$ the mean in $[c_j, c_{j+1})$, and $\beta_j$ the average increase relative to the first bin.

```
coef(lm(wage ~ cut(age, 4), data = Wage))
```

```
##           (Intercept)   cut(age, 4)(33.5,49]   cut(age, 4)(49,64.5]
##             94.158392              24.053491              23.664559
## cut(age, 4)(64.5,80.1]
##              7.640592
```

Step functions miss trends within bins unless the cutpoints happen to be in the right places.

## 10.3 Regression splines

**Motivation.** Polynomials are smooth but global: a change in one region moves the whole curve. Step functions are local but jumpy. Splines combine the two virtues, fitting low-degree polynomials locally and joining them smoothly.

**Basis functions.** Polynomial and step regression are special cases of

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_K b_K(x_i) + \epsilon_i$$

with fixed, known basis functions chosen in advance: $b_j(x) = x^j$ for polynomials, $b_j(x) = I(c_j \le x < c_{j+1})$ for step functions. This is a standard linear model in the predictors $b_j(x_i)$; again we look at the fitted curve, not the coefficients.

**Piecewise polynomials.** Fit separate low-degree polynomials on different regions of $X$; the points where the coefficients change are the **knots**. A piecewise cubic with one knot at $c$:

$$y_i = \begin{cases} \beta_{01} + \beta_{11} x_i + \beta_{21} x_i^2 + \beta_{31} x_i^3 + \epsilon_i, & x_i < c, \\ \beta_{02} + \beta_{12} x_i + \beta_{22} x_i^2 + \beta_{32} x_i^3 + \epsilon_i, & x_i \ge c. \end{cases}$$

Unconstrained, the two pieces do not meet at $c$. Requiring continuity at $c$, and then continuity of the first and second derivatives, gives a curve that is continuous and smooth: a **cubic spline**. The left piece uses 4 degrees of freedom; the right piece 4 without constraints, 3 with continuity, 1 with the two derivative constraints as well. In general a cubic spline with $K$ knots uses $K+4$ degrees of freedom.

**Spline basis.** A cubic spline with $K$ knots can be written $y_i = \beta_0 + \beta_1 b_1(x_i) + \cdots + \beta_{K+3} b_{K+3}(x_i) + \epsilon_i$. The most direct basis starts from $x, x^2, x^3$ and adds one **truncated power function** per knot,

$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3, & x > \xi, \\ 0, & \text{otherwise.} \end{cases}$$

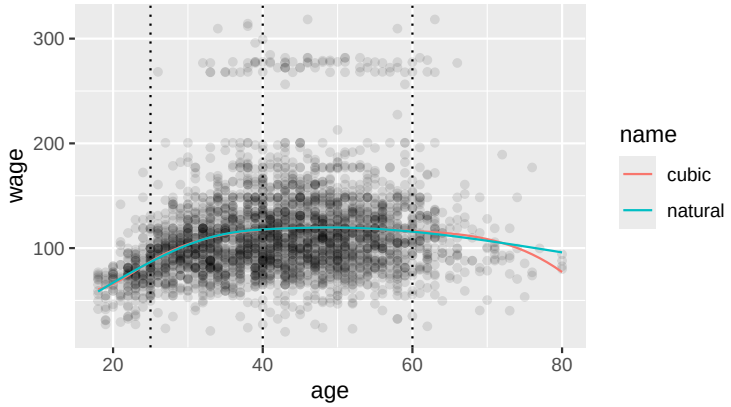
Adding $\beta h(x, \xi)$ to a cubic creates a discontinuity only in the third derivative at $\xi$, so the function and its first two derivatives stay continuous. With $K$ knots: intercept, $X, X^2, X^3$ and $h(X, \xi_1), \ldots, h(X, \xi_K)$, i.e. $K+4$ coefficients.

**Why the truncated power basis works [Supplement]**

$h(x,\xi)$ and its first two derivatives, $3(x-\xi)_+^2$ and $6(x-\xi)_+$, vanish at $x=\xi$ from both sides, while the third derivative jumps from 0 to 6. Conversely, any cubic spline with knots $\xi_1 < \cdots < \xi_K$ differs on $[\xi_k, \xi_{k+1})$ from its expression on $[\xi_{k-1}, \xi_k)$ by a cubic that vanishes with two derivatives at $\xi_k$, which must be $c\,(x-\xi_k)^3$. So the $K+4$ functions span exactly the cubic splines.

In R, `bs()` gives a numerically better-behaved basis (B-splines) for the same space:

```
fit_bs <- lm(wage ~ bs(age, knots = c(25, 40, 60)), data = Wage)
fit_ns <- lm(wage ~ ns(age, df = 4), data = Wage)
grid <- tibble(age = 18:80) %>%
  mutate(cubic = predict(fit_bs, .), natural = predict(fit_ns, .))
ggplot(Wage, aes(age, wage)) + geom_point(alpha = 0.1) +
  geom_line(data = pivot_longer(grid, -age), aes(y = value, colour = name)) +
  geom_vline(xintercept = c(25, 40, 60), linetype = "dotted")
```




**Natural splines.** Splines have high variance near the ends of the range of $X$: the confidence bands there look wild. A **natural spline** adds boundary constraints, requiring the function to be linear beyond the boundary knots, which gives more stable estimates there (`ns()`).

**Number and location of knots.** Place more knots where the function might vary most rapidly; in practice knots are placed uniformly, e.g. at quantiles of $X$ (`ns(age, df = 4)` does this). How many? Trying several and keeping the best-looking curve is subjective; cross-validation chooses the number that gives the smallest held-out RSS.

```
set.seed(1)
folds <- sample(rep(1:10, length.out = nrow(Wage)))
cv_ns <- function(df) mean(map_dbl(1:10, function(k) {
  fit <- lm(wage ~ ns(age, df = df), data = Wage[folds != k, ])
  mean((Wage$wage[folds == k] - predict(fit, Wage[folds == k, ]))^2)
}))
round(map_dbl(1:8, cv_ns), 1)
```

```
## [1] 1676.8 1602.4 1595.4 1595.2 1595.1 1594.0 1593.9 1593.6
```

CV error drops sharply up to about 3 degrees of freedom and is nearly flat after that. Compared with a polynomial of the same number of degrees of freedom, a natural spline is better behaved at the boundaries, because it uses knots rather than high powers to gain flexibility.

## 10.4 Smoothing splines

**Motivation.** Choosing knots is awkward. Smoothing splines avoid the choice by putting a knot at every data point and controlling wiggliness with a penalty instead, the same loss-plus-penalty idea as ridge regression.

We want a function $g$ with small RSS $= \sum_i (y_i - g(x_i))^2$, but without constraints $g$ can interpolate the data and make RSS zero (overfitting). We want $g$ small in RSS *and* smooth. The **smoothing spline** minimises

$$\sum_{i=1}^{n} \left(y_i - g(x_i)\right)^2 + \lambda \int g''(t)^2 \, dt, \qquad \lambda \geq 0.$$

This is "loss + penalty": $g''$ measures roughness, since a smooth $g$ has nearly constant slope and small $\int g''^2$, while a jumpy $g$ has a wildly varying slope. With $\lambda = 0$ the penalty has no effect and $g$ interpolates the training data; as $\lambda \to \infty$, $g$ becomes a straight line. So $\lambda$ controls the bias–variance trade-off. The minimiser is a natural cubic spline with knots at every unique $x_i$, shrunk by the penalty (the slides note the $\lambda = 0$ case).

**Effective degrees of freedom.** The fitted values are linear in $y$: $\hat{g}_\lambda = S_\lambda y$ for an $n \times n$ smoother matrix $S_\lambda$. The **effective degrees of freedom** are

$$\mathrm{df}_\lambda = \sum_{i=1}^{n} \{S_\lambda\}_{ii},$$

which decreases from $n$ to 2 as $\lambda$ goes from 0 to $\infty$ and measures flexibility. For least squares $S$ is the hat matrix and its trace is the number of parameters, which motivates the definition [Supplement].

**Choosing $\lambda$.** No knots need to be chosen, but $\lambda$ must be. LOOCV is cheap:

$$\mathrm{RSS}_{\mathrm{cv}}(\lambda) = \sum_{i=1}^{n} \left(y_i - \hat{g}_\lambda^{(-i)}(x_i)\right)^2 = \sum_{i=1}^{n} \left[\frac{y_i - \hat{g}_\lambda(x_i)}{1 - \{S_\lambda\}_{ii}}\right]^2,$$

where $\hat{g}_\lambda^{(-i)}$ is fitted without observation $i$; a single fit gives all $n$ leave-one-out errors.

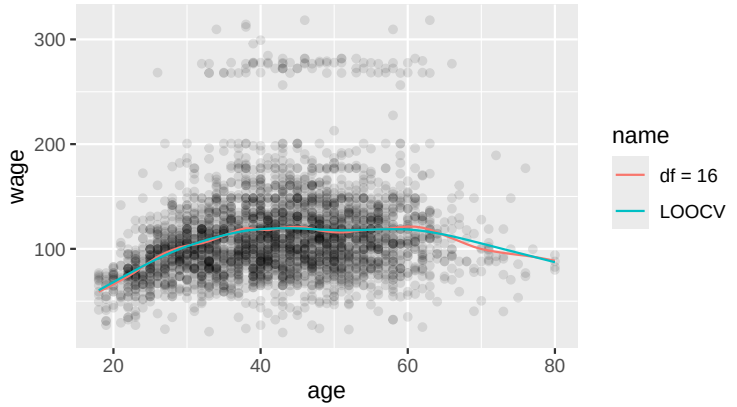
```
ss16 <- smooth.spline(Wage$age, Wage$wage, df = 16)
ss_cv <- smooth.spline(Wage$age, Wage$wage, cv = TRUE)
```

```
## Warning in smooth.spline(Wage$age, Wage$wage, cv = TRUE): cross-validation with non-unique 'x'
##     values seems doubtful
```

```
c(df_chosen = ss_cv$df, lambda = ss_cv$lambda)
```

```
##  df_chosen     lambda
## 6.79459570 0.02792303
```

```
tibble(age = 18:80, `df = 16` = predict(ss16, 18:80)$y,
       `LOOCV` = predict(ss_cv, 18:80)$y) %>%
  pivot_longer(-age) %>%
  ggplot(aes(age, value, colour = name)) +
  geom_point(data = Wage, aes(age, wage), inherit.aes = FALSE, alpha = 0.1) +
  geom_line() + labs(y = "wage")
```

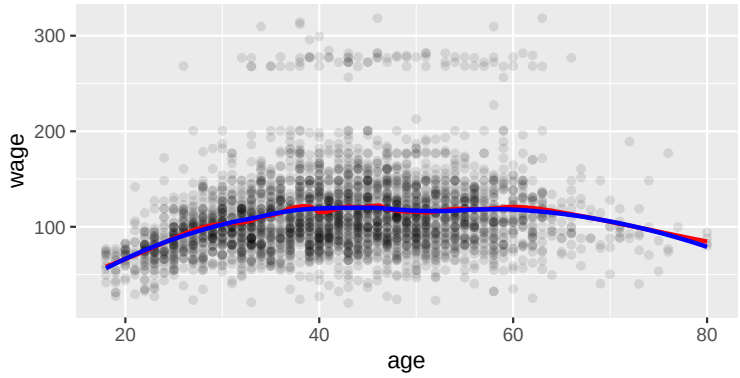



The LOOCV choice (about 7 effective degrees of freedom) is smoother than the 16-df fit and almost as good on the data.

## 10.5 Local regression

Local regression (LOESS, Lecture 2) fits at each target point $x_0$ a weighted least-squares line (or quadratic) using the fraction $s$ (the **span**) of points nearest to $x_0$, with weights decreasing with distance. The span plays the role of $\lambda$: a smaller span gives a more wiggly fit. It is memory-based (all data are needed at prediction time) and works poorly in more than three or four dimensions.

```
ggplot(Wage, aes(age, wage)) + geom_point(alpha = 0.1) +
  geom_smooth(method = "loess", span = 0.2, se = FALSE, colour = "red", formula = y ~ x)
     +
  geom_smooth(method = "loess", span = 0.5, se = FALSE, colour = "blue", formula = y ~ x
    )
```




## 10.6 Generalized additive models

**Motivation.** All the smoothers so far handle one predictor. GAMs combine them: each predictor gets its own smooth function, and the functions are added, keeping the interpretability of an additive model.

A GAM replaces each linear term $\beta_j x_{ij}$ of the multiple regression model by a smooth function:

$$y_i = \beta_0 + \sum_{j=1}^{p} f_j(x_{ij}) + \epsilon_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \epsilon_i.$$

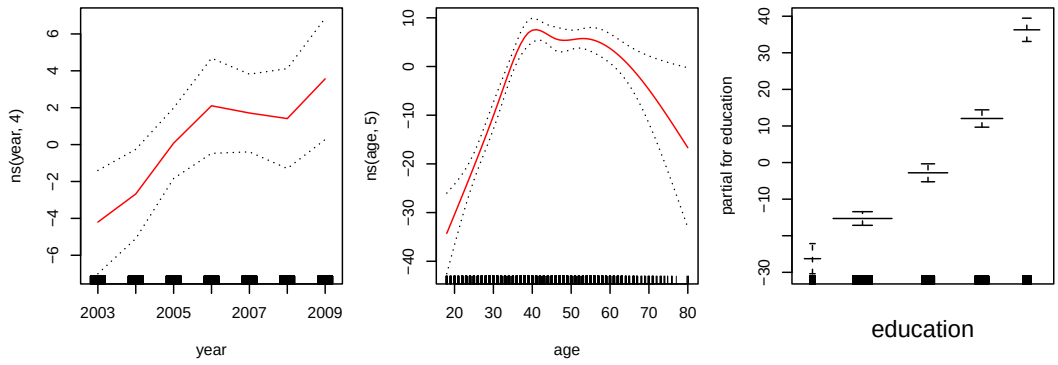
It is *additive* because a separate $f_j$ is computed for each $X_j$ and the contributions are added. The single-variable methods above are the building blocks, and GAMs work for quantitative and qualitative responses.

For the wage data,

$$\text{wage} = \beta_0 + f_1(\text{year}) + f_2(\text{age}) + f_3(\text{education}) + \epsilon,$$

with natural splines for year and age and a separate constant for each of the five education levels. Since natural splines are basis expansions, the whole model is one big regression on spline bases and dummy variables, fitted by `lm()`:

```
gam1 <- lm(wage ~ ns(year, 4) + ns(age, 5) + education, data = Wage)
par(mfrow = c(1, 3), mar = c(4, 4, 1, 1))
plot.Gam(gam1, se = TRUE, col = "red")
```




Each panel shows one $f_j$ with the others held fixed: with age and education fixed, wage rises slightly with year; with education and year fixed, wage is highest in middle age; with age and year fixed, wage rises with education. Smoothing splines can be used instead (`gam::gam(wage ~ s(year, 4) + s(age, 5) + education)`, fitted by backfitting).

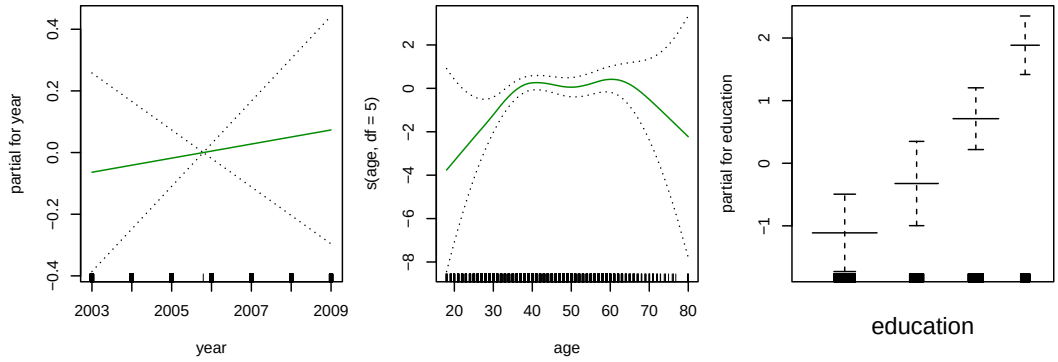
**Pros.** A nonlinear $f_j$ is fitted to each $X_j$ automatically, without trying transformations by hand; the fits can predict more accurately; additivity lets us examine each $X_j$'s effect with the others fixed; the smoothness of each $f_j$ is summarised by its degrees of freedom. **Cons.** The model is restricted to be additive, so important interactions can be missed. Interactions can be added by hand, as products $X_j \times X_k$ or as low-dimensional smooth functions $f_{jk}(X_j, X_k)$ (two-dimensional smoothers); fully general models need more flexible approaches such as random forests and boosting.

**GAMs for classification.** For binary $Y$ with $p(X) = P(Y = 1 \mid X)$, the logistic GAM is

$$\log \frac{p(X)}{1 - p(X)} = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p).$$

For the probability of earning more than \$250 000:

```
gam_lr <- gam(I(wage > 250) ~ year + s(age, df = 5) + education,
              family = binomial, data = Wage, subset = (education != "1. < HS Grad"))
par(mfrow = c(1, 3), mar = c(4, 4, 1, 1))
plot(gam_lr, se = TRUE, col = "green4")
```




The lowest education level is excluded because it contains no high earners, which would make its coefficient diverge (Lecture 9's separation problem) [Supplement].

**Summary.** Regression splines use basis functions; smoothing splines use loss plus penalty; GAMs extend linear models by replacing $x_j$ with $f_j(x_j)$ while keeping additivity, and can use splines as building blocks.